
初期粒子の発生方法

Geant4 11.3 準拠

Geant4講習会資料: HEP/NP/Space 2025



KEK

大学共同利用機関法人

高エネルギー加速器研究機構

本資料に関する注意

- 本資料の知的所有権は、高エネルギー加速器研究機構およびGeant4 collaborationが有します
- 以下のすべての条件を満たす場合に限り無料で利用することを許諾します
 - 学校、大学、公的研究機関等における教育および非軍事目的の研究開発のための利用であること
 - Geant4の開発者はいかなる軍事関連目的へのGeant4の利用を拒否します
 - このページを含むすべてのページをオリジナルのまま利用すること
 - 一部を抜き出して配布したり利用してはいけません
 - 誤字や間違いと疑われる点があれば報告する義務を負うこと
- 商業的な目的での利用、出版、電子ファイルの公開は許可なく行えません

目次

1. 初期粒子(Primary Particles)発生の基本
2. 初期粒子(Primary Particles)発生の方法
 - 手法#1: ツールキット提供のParticle Generatorsの使用
 - 手法#2: ユーザが独自に作るParticle Generatorの使用

初期粒子(Primary Particles)発生**の**基礎

初期粒子とその発生方法

- 一つの事象をシミュレーションする際に、初期条件として与える粒子を初期粒子 (*primary particle*) とよぶ
- primary particleの発生は、ユーザが行わなければならない**必須のアクション**
- primary particleを発生させるには大きく分けて以下の**2つの方法**がある

手法 #1: Geant4が提供している*Build-in Primary Particle Generators*を使用

- ユーザにとって**手軽な方法**
- 代表的なBuild-in Primary Particle Generators:
 - G4ParticleGun
 - G4GeneralParticleSource (GPS)

手法 #2: ユーザが独自のprimary particle generatorを実装

- ユーザにとって**自由度の高い方法**
 - Build-in Primary Particle Generatorsでは表現が難しい発生条件を実現可能

- primary particleの発生手法は、**上の二つの方法で同一**：

ステップ #1: Primary Particle Generatorの作成

- primary particle generatorクラスを作り、その中に初期粒子を発生させる手法(**上記、手法#1あるいは手法#2のいずれか**)を記述

ステップ #2: 作成したPrimary Particle Generatorの登録

- 上で作ったParticle GeneratorクラスのオブジェクトをG4RunManagerに渡す

Primary Particleの発生手法 ステップ#1: Primary Particle Generatorの作成

■ 作成方法は以下の2つのサブ・ステップからなる:

サブ・ステップ #1:

- *G4VUserPrimaryGeneratorAction*を継承し、ユーザのPrimary Generatorのクラス定義を行う

```
#include "G4VUserPrimaryGeneratorAction.hh"
class G4Event;
```

```
class PrimaryGenerator : public G4VUserPrimaryGeneratorAction
{
public:
    PrimaryGenerator();
    virtual ~PrimaryGenerator();

public:
    virtual void GeneratePrimaries(G4Event*);
};
```

Primary Generatorの
ユーザ・クラスを定義

[注] クラス名は任意

この関数の中に初期粒子
発生の手法を記述する

サブ・ステップ #2:

- ユーザは初期粒子を発生させる手法(前スライドの手法#1あるいは手法#2のいずれか)を *GeneratePrimaries()*関数の中に記述する
➡ この関数の具体的な書き方はこの講義の後半で説明
- *GeneratePrimaries()*関数内で発生させたprimary particle情報は*G4Event*オブジェクトに付加され、*G4RunManager*に渡される

Primary Particleの発生手法 ステップ#2: Primary Particle Generatorの登録

- 前のステップで作成したPrimary Particle Generatorのオブジェクトをユーザ・アクション初期化クラスに登録した後、G4RunManagerに渡すことで、初期粒子の発生方法の設定が完了
- ユーザ・アクション初期化クラスへの登録方法は以下の2つのサブ・ステップからなる:

サブ・ステップ #1:

- G4VUserActionInitializationクラスを継承し、ユーザ・アクション初期化クラスを定義する

```
class UserActionInitialization : public G4VUserActionInitialization
{
public:
    UserActionInitialization();
    virtual ~UserActionInitialization() override;

public:
    virtual void Build() const override;
};
```

← ユーザ・アクション初期化のクラスを定義
[注] クラス名は任意

← Primary Particle Generatorを含む、全てのユーザ・アクションの登録はこの関数内で行う

➡ ユーザ・アクション初期化クラスは一回、定義するだけでOK

サブ・ステップ #2:

- 上のBuild()関数に前ステップ1で定義したPrimaryGeneratorクラスのオブジェクトを登録

```
void UserActionInitialization::Build() const
{
    SetUserAction{ new PrimaryGenerator };
}
```

上記クラス定義の実装ファイル

← PrimaryGeneratorをユーザ・アクションの一つとして登録

[注]

- PrimaryGeneratorを含む全ユーザ・アクション(例:G4EventAction) の登録は上のユーザ・アクション初期化クラスのBuild()関数内でまとめて行うこと
- これで登録された全てのユーザ・アクションは、上のユーザ・アクション初期化クラスのオブジェクトをG4RunManagerに渡すことで設定が完了する (次のスライド参照)

ユーザ・アクションの設定

■ ユーザ・アクション設定の全体の流れは以下となる:

- ① ユーザ・アクション クラスを定義する
- ② ユーザ・アクション初期化 クラスを定義し、その中でユーザ・アクション クラスを登録
- ③ ユーザ・アクション初期化 クラスのオブジェクトをG4RunManagerに渡す

UserActionInitialization.cc

```
//+++++
// UserActionInitialization.cc
//+++++
#include "UserActionInitialization.hh"
#include "PrimaryGenerator.hh"

//-----
UserActionInitialization::UserActionInitialization()
: G4UserActionInitialization()
//-----
{}

//-----
UserActionInitialization::~UserActionInitialization()
//-----
{}

//-----
void UserActionInitialization::Build() const
//-----
{
    SetUserAction( new PrimaryGenerator{} );
    SetUserAction( new UserRunAction{} );
    SetUserAction( new UserSteppingAction{} );
}
```

PrimaryGeneratorおよび他のユーザアクションの
UserActionInitializationへの登録

UserActionInitializationのRunManagerへの登録

Application_Main.cc

```
//+++++
// Geant4 Application: Tutorial course for Hep/Medicine Users
//+++++
#include "Geometry.hh"
#include "UserActionInitialization.hh"

#include "G4RunManager.hh"
#include "G4UIExecutive.hh"
#include "FTFP_BERT.hh"

//-----
int main( int argc, char** argv )
//-----
{
    // Construct a run manager
    G4RunManager* runManager = new G4RunManager{};

    // Set up mandatory user initialization: Geometry
    runManager->SetUserInitialization( new Geometry{} );

    // Set up mandatory user initialization: Physics-List
    runManager->SetUserInitialization( new FTFP_BERT{} );

    // Set up user initialization: User Actions
    runManager->SetUserInitialization( new UserActionInitialization{} );

    // Initialize G4 kernel
    runManager->Initialize();

    // Start interactive session
    G4UIExecutive* uiExec = new G4UIExecutive{ argc, argv, "tcsh" };
    uiExec->SessionStart();

    // Job termination
    delete uiExec;
    delete runManager;

    return 0;
}
```

[注]

この講義ではユーザ・アクション定義方法の基本的な流れを説明。具体的な扱いはHands-onで学ぶ。

初期粒子(Primary Particles)の発生方法

手法#1: ツールキット提供のParticle Generatorsの使用

G4ParticleGunをPrimary Particles Generatorとして使う

- **G4ParticleGun**はGeant4標準提供の手軽なPrimary Particles Generatorの一つ
- G4ParticleGunは一度に**一種類の粒子だけ**を発生させる
- 発生させるprimary particleに対してユーザが設定可能なパラメータ
 - 粒子の種類／電荷
 - エネルギー／運動量
 - 偏極率
 - 発射位置／発射時間
- 使い方は基本的に以下の二つに分けられる：
 - UIコマンドを用いて発生粒子をコントロールする
 - G4VUserPrimaryGeneratorActionクラスのGeneratePrimaries()の中でG4ParticleGunのメンバー関数を用いて粒子発生をコントロール

[注]

- 上記設定パラメータが全て同じ粒子は、一度に複数個発生可能
- 異なったパラメータの粒子を複数個発生させたいければG4ParticleGunを複数用いる
(この手法はHands-onで学ぶ)

G4ParticleGunの設定

- Primary Particles Generatorの設定はG4VUserPrimaryGeneratorActionを継承したユーザ・クラス(ここではPrimaryGeneratorという名前を使う)で行う ➡ 本講義スライドp.6参照

本講義スライドp.6で示したコードの一部

```
class PrimaryGenerator : public G4VUserPrimaryGeneratorAction{
public:
    PrimaryGenerator();
    virtual ~PrimaryGenerator();

public:
    virtual void GeneratePrimaries(G4Event*); .....
```

- G4ParticleGunをPrimary Particles Generatorとして設定するのは以下の2ステップで完了
ステップ #1: 上記クラス定義内のコンストラクタでG4ParticleGunを作成

```
PrimaryGenerator::PrimaryGenerator {
    fpParticleGun = new G4ParticleGun{};
}
```

[注] fpParticleGunはPrimaryGeneratorクラスのデータメンバーとして定義しておく

- ステップ #2: 上記クラス定義内のGeneratePrimaries()関数を以下のように実装

```
void PrimaryGenerator::GeneratePrimaries(G4Event* anEvent) {
    fpParticleGun-> GeneratePrimaryVertex(anEvent);
}
```

[注] この設定の後、スライドp.7で述べたPrimary Particle Generatorの登録を行えばGeneratorは使用可能

G4ParticleGunの使い方

- 直前のスライドでの手続きをすれば、次のようなUIコマンドを使い初期粒子を発生できる

/gun/list	使用可能な粒子の種類表示
/gun/particle	発生させる粒子の種類設定
/gun/direction	粒子発生角度(運動量方向ベクトル)設定
/gun/energy	粒子の運動エネルギー設定
/gun/momentum	粒子の運動量設定
/gun/momentumAmp	粒子の運動量絶対値設定
/gun/position	粒子の発生点設定
/gun/time	粒子の発生時間設定
/gun/polarization	粒子の偏極率設定
/gun/number	事象あたりの発生粒子数の設定(同一物理条件の粒子)
/gun/ion	イオン発生条件設定 [使用方法] /gun/ion Z A Q

- また、*G4ParticleGun*クラス提供の関数を用いることで、初期粒子をプログラム内で発生できる

```
SetNumberOfParticles(G4int)
SetParticleDefinition(G4ParticleDefinition*)
SetParticleMomentum(G4ParticleMomentum)
SetParticleMomentumDirection(G4ThreeVector)
SetParticleEnergy(G4double)
SetParticleTime(G4double)
SetParticlePosition(G4ThreeVector)
SetParticlePolarization(G4ThreeVector)
```

[注] これらの関数を用いて、UIコマンドでは設定できない事象初期条件を記述することができる:

- (例)
- 同一点から発生する複数の粒子(粒子の種類、エネルギー、方向等が異なる)
 - 複数の発生点(primary vertex)を持つ事象

General Particle Source (GPS)をPrimary Particles Generatorとして使う

- `G4GeneralParticleSource(GPS)`はGeant4標準提供のPrimary Particles Generatorの一つ
- 先に述べたG4ParticleGunの機能を拡張したもの
 - 例えば、G4ParticleGunが発生する粒子の発生点(primary vertex)は点であったが、GPSでは発生点に広がりを持たすことができる
 - GPSの”Source(ソース)”の意味は、発生点が放射線源のように広がりを持つということ
- GPSのソースはユーザが指定する以下のパラメータに従ってprimary particlesを発生させる
 - ソースの形状設定
 - ・ 点、平面、立体、 その他
 - 粒子のエネルギーおよび角度分布設定
 - ・ 一様、ガウス、指数関数、その他 (built-in)
 - ・ ユーザが設定
 - 複数のソース設定 — 相対強度を設定できる
- GPSはUIコマンドを用いてのみ使用する
 - ➡ G4ParticleGunが提供するようなプログラム関数はない

GPSの設定

- Primary Particles Generatorの設定はG4VUserPrimaryGeneratorActionを継承したユーザ・クラス(ここではPrimaryGeneratorという名前を使う)で行う ➡ 本講義スライドp.6参照

本講義スライドp.6で示したコードの一部

```
class PrimaryGenerator : public G4VUserPrimaryGeneratorAction{
public:
    PrimaryGenerator();
    virtual ~PrimaryGenerator();

public:
    virtual void GeneratePrimaries(G4Event*); .....
```

- G4GeneralParticleSourceをPrimary Particles Generatorと設定するのは以下の2ステップで完了
ステップ #1: 上記クラス定義中のコンストラクタでG4GeneralParticleSourceを作成

```
PrimaryGenerator::PrimaryGenerator {
    fpGPS = new G4GeneralParticleSource{};
}
```

[注] fpGPSはPrimaryGeneratorクラスのデータメンバーとして定義しておく

- ステップ #2: 上記クラス定義中のGeneratePrimaries()関数を以下のように実装

```
void PrimaryGenerator::GeneratePrimaries(G4Event* anEvent) {
    fpGPS-> GeneratePrimaryVertex(anEvent);
}
```

[注] この設定の後、スライドp.7で述べたPrimary Particle Generatorの登録を行えばGeneratorは使用可能

GPSの使い方

- 直前のスライドでの手続きをすれば、GPSのUIコマンドを使い初期粒子を発生できる

- コマンドおよび使用例はユーザマニュアル 'Book For Application Developers' の以下の章を参照:

Getting Started with Geant4 - Running a Simple Example

-> Geant4 General Particle Source

 具体的な使用方法是Hands-onで学ぶ

[注]

GPSはG4ParticleGunの機能拡張版で機能は豊富であるが、ユーザの要求を全て満足させる一般性はない。自由度の高い初期粒子発生が必要なら、次節の手法を用いることを推奨。

初期粒子(Primary Particles)の発生方法

手法#2: ユーザが独自に作るParticle Generatorの使用

ユーザが独自にPrimary Particles Generatorを作る

- G4ParticleGun及びG4GeneralParticleSourceは手軽に初期粒子を発生させるツール
- 一方、ユーザが自らのアプリに特化した初期粒子発生手法を使いたい場合がある
- ここでは、ユーザが独自のprimary particles generatorを作成し、設定する手法を述べる
- 作成手法の流れはG4ParticleGunおよびGPSと全く同じで、以下の継承クラスの **GeneratePrimaries()** 関数に発生手法を書くだけ

本講義スライド#6で示したコードの一部

```
class PrimaryGenerator : public G4VUserPrimaryGeneratorAction{
public:
    PrimaryGenerator();
    virtual ~PrimaryGenerator();

public:
    virtual void GeneratePrimaries(G4Event*); .....
```

- G4ParticleGunの場合: **GeneratePrimaries()**関数を次のように記述した

```
void PrimaryGenerator::GeneratePrimaries(G4Event* anEvent) {
    fpParticleGun-> GeneratePrimaryVertex(anEvent);
}
```

← G4ParticleGunが提供している関数を使い、初期粒子発生をしている

- ユーザ独自の発生方法の場合: **GeneratePrimaries()**関数を次のように書けば良い

```
void PrimaryGenerator::GeneratePrimaries(G4Event* anEvent) {
    // ---- 初期粒子発生コードをユーザがここに書き、初期粒子発生させる ----
}
```

➡ **GeneratePrimaries()**関数の具体的な書き方は次のスライドで述べる

ユーザ独自の初期粒子発生コードの書き方

■ ユーザがGeneratePrimaries(G4Event*)関数を記述するには以下の3つのクラスを使用する

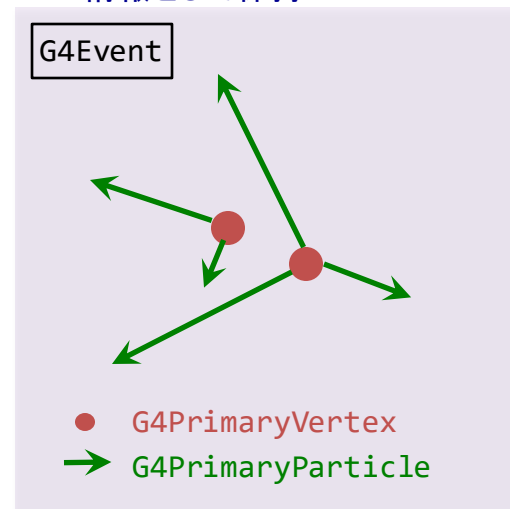
- **G4PrimaryParticle** (初期粒子発生の際のみに使われるクラス)
 - このクラスのオブジェクトを一つ作れば、一つの初期粒子が生成できる
 - 初期粒子の物理情報(粒子の種類、運動量、など)はクラスのコンストラクタで指定する
 - 初期粒子の発生点の位置情報は以下のG4PrimaryVertexを使って指定する
- **G4PrimaryVertex** (初期粒子発生の際のみに使われるクラス)
 - このクラスのオブジェクトを一つ作れば、初期粒子の発生点(vertex)が一つ生成できる
 - vertexの物理情報(位置座標、発生時間、など)はクラスのコンストラクタで指定する
 - 一つのvertexオブジェクトに任意の数の初期粒子を持たせることができる
 - 一つの事象(event)に任意の数のvertexを持たせることができる
- **G4Event**
 - このクラスのオブジェクトは一つの事象(event)の全体情報を保持している
 - 初期粒子の情報はeventの全体情報の一つであり、eventはそれをvertex情報として保持

■ GeneratePrimaries()関数は次の3つのステップで記述できる

- ① **G4PrimaryParticle**クラスのコンストラクタを使い、発生させたい初期粒子を全て作る
- ② **G4PrimaryVertex**クラスのコンストラクタでvertex(複数可能)を作り同じ発生点を持つ初期粒子は一つのvertexに'AddPrimary()'関数を使ってまとめる
- ③ **G4Event**クラスのオブジェクトがGeneratePrimaries()の引数なので、'AddPrimaryVertex()'関数を使い、作成したvertexオブジェクトをeventオブジェクトに加えることでGeneratePrimaries()関数記述が完了

[注] G4ParticleGunもG4GeneralParticleSourceこのステップで記述されている

- 実装の具体例はHands-onで学ぶ



講義 終了